FUNCTIONAL TESTING

Software Testing – Project Report

Name: Malathi

Functional testing is a software testing method used to verify whether an application performs its required functions correctly. It focuses on what the system does rather than how the internal code is written.

The main purpose is to check that each feature works according to the specified requirements. Testers provide inputs, observe outputs, and compare the actual behavior with the expected behavior.

Functional testing can be performed at different levels, including unit, integration, system, and acceptance testing. It is an important part of software quality assurance.

Prepared for: Malathi

Objectives of Functional Testing

The primary objective is to verify that all functional requirements are implemented correctly. Testing should cover normal conditions as well as invalid and boundary inputs.

Important objectives include checking user interactions, validating business rules, verifying data processing, confirming error handling, and ensuring that expected outputs are produced.

Functional testing also helps identify missing features, incorrect calculations, broken workflows, and unexpected system responses before software is released.

Types of Functional Testing

Common types include unit testing, integration testing, system testing, smoke testing, sanity testing, regression testing, and user acceptance testing.

Unit testing checks individual components. Integration testing verifies communication between modules. System testing evaluates the complete application against requirements.

Smoke testing checks whether a build is stable enough for detailed testing. Sanity testing focuses on specific changes. Regression testing ensures that existing functionality has not been broken by new modifications.

Functional Test Cases

A functional test case describes a specific condition to be tested. It normally contains a test case ID, feature name, preconditions, test steps, input data, expected result, actual result, and status.

Example: For a login feature, enter a valid username and password and select Login. The expected result is successful authentication and navigation to the appropriate page.

Negative test cases are equally important. For example, entering an incorrect password should display an appropriate error message without granting access.

Test Case	Input / Action	Expected Result
TC-01	Valid input	Function succeeds
TC-02	Invalid input	Error is displayed
TC-03	Boundary input	Correct boundary behavior

Functional Testing Process

The process begins by understanding requirements and identifying the functions that need verification. Test scenarios are then created for each important feature.

Next, test data and test cases are prepared. Testers execute the cases, record actual results, and compare them with expected results.

When a failure is found, a defect is reported with sufficient information for developers to reproduce and fix the issue. After correction, the affected functionality is retested.

Black-Box Testing Techniques

Functional testing is commonly associated with black-box testing because testers can validate behavior without knowing the internal implementation.

Important techniques include equivalence partitioning, boundary value analysis, decision table testing, state transition testing, and use-case testing.

Equivalence partitioning divides input data into groups that are expected to behave similarly. Boundary value analysis focuses on values at and around limits, where defects are often found.

Tools Used for Functional Testing

Functional testing can be performed manually or with automation tools. Manual testing is useful for exploratory checks and user-interface scenarios.

Automation tools can execute repetitive test cases quickly and consistently. Examples include Selenium for web testing, Cypress for modern web applications, and Postman for API testing.

The choice of tool depends on application type, technology stack, testing goals, team skills, and maintenance requirements.

Test Case	Input / Action	Expected Result
TC-01	Valid input	Function succeeds
TC-02	Invalid input	Error is displayed
TC-03	Boundary input	Correct boundary behavior

Advantages and Limitations

Advantages include requirement validation, early defect detection, improved reliability, better user experience, and reduced risk before deployment.

Automated functional tests can save time when the same scenarios need to be executed repeatedly. They also provide consistent execution and useful test reports.

Limitations include test maintenance, the effort required to create comprehensive test cases, and the possibility of missing unexpected behavior if testing is limited only to predefined scenarios.

Functional Testing Example

Consider an online learning application with a login, course search, enrollment, and quiz feature. Functional testing verifies that users can log in, search for courses, enroll successfully, and submit quizzes.

Sample scenarios: valid login should succeed; invalid login should show an error; a matching course search should display relevant results; enrollment should update the learner's course list; quiz submission should calculate and display the appropriate result.

These scenarios demonstrate how functional testing validates complete user workflows.

Test Case	Input / Action	Expected Result
TC-01	Valid input	Function succeeds
TC-02	Invalid input	Error is displayed
TC-03	Boundary input	Correct boundary behavior

Conclusion

Functional testing is essential for confirming that software features work according to requirements. It evaluates inputs, processing, outputs, and user-visible behavior.

A structured approach using clear requirements, well-designed test cases, suitable test data, defect tracking, and regression testing improves software quality.

Functional testing should be integrated throughout the software development lifecycle rather than being treated as only a final activity.

Name: Malathi